

# claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

## Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-inde  
xer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_58a19e3e-3ce\agent-  
a05e916c5cf95af04.jsonl`  
- SHA-256: `b30314ff0e38227a4930d488d5dcb3eed79b686db2e92086c04581a056601fec`  
- Source modified: `2026-07-06T23:21:58+00:00`  
- Imported at: `2026-07-08T16:00:05+00:00`  
- project: `wf_58a19e3e-3ce`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

## Transcript

### 1. user (2026-07-06T23:19:54.509Z)

Reviewing GitHub PR #54 "feat(policy): add F5 deep inspection budget boundary" (branch feat/f5-deep-inspection-budget -> main). ADR 0012 F5.  
+364/-4, 3 files. Checked-out copy at C:/Users/avidu/Projects/\_wt-pr54 (read there or 'git -C C:/Users/avidu/Projects/Telegram show origin/feat/f5-deep-inspection-budget:<path>').  
Run repros: PYTHONPATH=C:/Users/avidu/Projects/\_wt-pr54/src  
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script>.

WHAT F5 DOES (policy.py) ? BOUNDARY ONLY, no downloader/ffprobe/hash/scan (explicitly deferred):

```
- PolicyBudget(frozen dataclass): max_bytes_per_item, timeout_s, max_concurrent, + private  
_semaphore built in __post_init__ (via object.__setattr__)  
  when max_concurrent>0. async inspect(fetcher, content_item): 'async with self._semaphore:  
return await asyncio.wait_for(fetcher.inspect(item,self), timeout=timeout_s)'.  
- ContentFetcher(Protocol): async inspect(content_item, budget)->StageResult.  
PolicyContext.fetcher/budget now typed (were object|None in F1).  
- load_deep_inspection_enabled(policy): deep_inspect bool OR {enabled: bool}; invalid ->  
ValueError -> stage fail-OPEN (pass, invalid_deep_inspection_config).  
- DeepInspectionStage (Tier 4, LAST after topic/url/caption): opt-out if not enabled -> pass.  
_deep_inspection_budget_result checks (in order, all  
  BEFORE calling fetcher): budget present, budget valid (max_bytes/timeout/max_concurrent all  
>0), content_item.size_bytes present, size<=max_bytes ->  
  returns a FAIL StageResult on any violation. Then if fetcher missing -> fail. Else  
budget.inspect(...) under semaphore+wait_for; TimeoutError -> fail 'budget exceeded'.  
- Live wiring (_evaluate_policy_stages) creates PolicyContext WITHOUT fetcher/budget (both  
None), so deep_inspect enabled -> stage fails 'budget missing'  
  -> video DROPPED (fail-closed). This is intentional/dormant (no /set deep_inspect command  
exists).
```

VERIFIED by main reviewer: gate green (543 passed @ 94.57%); budget checks run BEFORE the  
fetcher; deep\_inspect enabled-without-budget drops the item  
(fail-closed, safe direction); default (deep\_inspect unset) unaffected. Comprehensive tests  
(missing/invalid budget, missing size, size-exceeded, timeout,  
concurrency max\_active==1, missing fetcher, valid delegate).

Ground rules: READ code, cite file:line. Run repros. Report ONLY defects in THIS PR's diff. No  
praise. Concrete failure scenario, ranked.  
Severity: high (budget/timeout/concurrency BYPASS ? heavy work runs unbounded, or a real bug),  
medium (real latent/design), low (note/hygiene/intentional-fail-closed).

ADVERSARIALLY VERIFY this finding ? try to REFUTE it. Read cited code, run a repro with  
PYTHONPATH=C:/Users/avidu/Projects/\_wt-pr54/src  
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe.  
CONFIRMED only if the defect genuinely ships in THIS PR's diff and produces the described wrong  
behaviour. REFUTED if handled / can't occur /

intentional-and-safe / out-of-scope-deferred. PLAUSIBLE if unsettled. Corrected severity (a real budget bypass = high; intentional fail-closed note = low).

FINDING (budget\_enforcement): Fetcher exceptions other than TimeoutError propagate uncaught out of DeepInspectionStage and evaluate\_pipeline

severity medium | src/tg\_video\_filter/policy.py:708

summary: DeepInspectionStage.evaluate wraps ctx.budget.inspect(...) in a try/except that catches only TimeoutError. Any other exception raised by the delegated fetcher.inspect (network error, ffmpeg/subprocess crash, ValueError, OSError, etc.) escapes the stage and evaluate\_pipeline entirely, unlike every other policy stage which fails-open by catching and returning a pass StageResult.

scenario: A wired fetcher raises RuntimeError('ffmpeg exploded') (or any non-timeout error) during deep inspection. The exception propagates out of evaluate() -> evaluate\_pipeline() -> \_evaluate\_policy\_stages(); the caller at telethon\_client.py:283 is not wrapped in try/except, so the message handler aborts mid-processing (the video is never recorded/delivered and the record insert at line 301-315 is skipped) rather than producing a clean fail-closed StageResult.

evidence: Repro confirmed: with a fetcher raising RuntimeError, both  
`DeepInspectionStage().evaluate(ctx)` and `evaluate\_pipeline(ctx,[stage])` print 'RAISED UNCAUGHT: RuntimeError ffmpeg exploded'. policy.py:708-717 catches only TimeoutError. No test in the PR (tests/test\_policy.py) exercises a non-TimeoutError fetcher exception. Dormant today because live wiring passes fetcher=None/budget=None (stage fails at 'budget missing' first), but active as soon as a real fetcher is added ? which is the whole point of this boundary.

rec: Decide the failure direction explicitly: either catch Exception around ctx.budget.inspect and return StageResult.fail(...) (fail-closed, consistent with dropping suspicious items) or StageResult.pass(...) with an error flag (fail-open, consistent with the other stages' config-error handling). Add a test asserting the chosen behavior so the contract is pinned before a fetcher lands.

## 2. assistant (2026-07-06T23:19:57.206Z)

I'll verify this finding by reading the cited code and running a repro.

## 3. user (2026-07-06T23:19:58.028Z)

```

650     *,
651     caption_present: bool,
652     caption_truncated: bool = False,
653     timed_out_patterns: tuple[str, ...] = (),
654 ) -> dict[str, object]:
655     metadata: dict[str, object] = {
656         "caption_present": caption_present,
657         "checked_terms": len(config.unsafe_terms),
658         "checked_patterns": len(config.unsafe_patterns),
659     }
660     if caption_truncated:
661         metadata["caption_truncated"] = True
662         metadata["max_caption_scan_chars"] = _MAX_CAPTION_SCAN_CHARS
663     if config.invalid_patterns:
664         metadata["ignored_invalid_caption_patterns"] = list(config.invalid_patterns)
665     if config.ignored_term_count:
666         metadata["ignored_caption_term_count"] = config.ignored_term_count
667     if config.ignored_pattern_count:
668         metadata["ignored_caption_pattern_count"] = config.ignored_pattern_count
669     if timed_out_patterns:
670         metadata["timed_out_caption_patterns"] = list(timed_out_patterns)
671     return metadata
672
673
674 class DeepInspectionStage:
675     """ADR 0012 Tier 4 opt-in boundary for budgeted heavy inspection."""
676
677     stage_name = "deep-inspection"
678
679     async def evaluate(self, ctx: PolicyContext) -> StageResult:
680         try:
681             enabled = load_deep_inspection_enabled(ctx.policy)

```

```

682         except (json.JSONDecodeError, ValueError) as exc:
683             log.warning(
684                 "deep inspection disabled due to invalid config policy_id=%s",
685                 ctx.policy.id,
686                 exc_info=True,
687             )
688             return StageResult.pass_(
689                 self.stage_name,
690                 f"invalid deep inspection config ignored: {exc}",
691                 invalid_deep_inspection_config=True,
692             )
693         if not enabled:
694             return StageResult.pass_(self.stage_name, "deep inspection disabled")
695
696         budget_result = _deep_inspection_budget_result(ctx)
697         if budget_result is not None:
698             return budget_result
699
700         if ctx.fetcher is None:
701             return StageResult.fail(
702                 self.stage_name,
703                 "deep inspection fetcher missing",
704                 missing_fetcher=True,
705             )
706
707         assert ctx.budget is not None
708         try:
709             return await ctx.budget.inspect(ctx.fetcher, ctx.content_item)
710         except TimeoutError:
711             return StageResult.fail(
712                 self.stage_name,
713                 "deep inspection budget exceeded",
714                 budget_exceeded=True,
715                 exceeded_budget="timeout_s",
716                 timeout_s=ctx.budget.timeout_s,
717             )
718
719
720 def _deep_inspection_budget_result(ctx: PolicyContext) -> StageResult | None:
721     budget = ctx.budget
722     if budget is None:
723         return StageResult.fail(
724             DeepInspectionStage.stage_name,
725             "deep inspection budget missing",
726             missing_budget=True,
727         )
728     if budget.max_bytes_per_item <= 0 or budget.timeout_s <= 0 or budget.max_concurrent
729     <= 0:
730         return StageResult.fail(
731             DeepInspectionStage.stage_name,
732             "deep inspection budget invalid",
733             invalid_budget=True,
734             max_bytes_per_item=budget.max_bytes_per_item,
735             timeout_s=budget.timeout_s,
736             max_concurrent=budget.max_concurrent,
737         )
738     size_bytes = ctx.content_item.size_bytes
739     if size_bytes is None:
740         return StageResult.fail(
741             DeepInspectionStage.stage_name,
742             "deep inspection budget cannot be enforced without content size",
743             missing_size_bytes=True,
744             max_bytes_per_item=budget.max_bytes_per_item,
745         )
746     if size_bytes > budget.max_bytes_per_item:

```

```

746         return StageResult.fail(
747             DeepInspectionStage.stage_name,
748             "deep inspection budget exceeded",
749             budget_exceeded=True,
750             exceeded_budget="max_bytes_per_item",
751             size_bytes=size_bytes,
752             max_bytes_per_item=budget.max_bytes_per_item,
753         )
754     return None
755
756
757     async def evaluate_pipeline(
758         ctx: PolicyContext,
759         stages: list[PolicyStage],
760     ) -> tuple[bool, list[StageResult]]:
761         """Evaluate stages in order and short-circuit on the first failure."""
762         results: list[StageResult] = []
763         for stage in stages:
764             result = await stage.evaluate(ctx)
765             results.append(result)
766             if not result.passed:
767                 return False, results
768         return True, results
769

```

#### 4. assistant (2026-07-06T23:20:00.429Z)

Now let me look at the caller and other stages to verify the fail-open claim, plus run the repro.

#### 5. user (2026-07-06T23:20:01.544Z)

```

400     except ValueError:
401         return None
402     host = parsed.hostname
403     if host is None:
404         return None
405
406     host = host.strip().strip("[]").rstrip(".").casefold()
407     if not host:
408         return None
409
410     try:
411         return ip_address(host).compressed.casefold()
412     except ValueError:
413         pass
414
415     try:
416         domain = host.encode("idna").decode("ascii").casefold()
417     except UnicodeError:
418         return None
419     if domain.startswith("www."):
420         domain = domain[4:]
421     if not _valid_domain_name(domain):
422         return None
423     return domain
424
425
426     def extract_caption_urls(text: str | None) -> tuple[ExtractedUrl, ...]:
427         """Extract plain URLs from captured caption text without network I/O."""
428         if not text:
429             return ()
430
431         urls: list[ExtractedUrl] = []
432         for match in _URL_RE.finditer(text):

```

```

433         raw_url = match.group(1).rstrip(_TRAILING_URL_PUNCTUATION)
434         while raw_url.endswith("(") and raw_url.count("(") < raw_url.count(")"):
435             raw_url = raw_url[:-1]
436         domain = normalize_domain(raw_url)
437         if domain is not None:
438             urls.append(ExtractedUrl(original_url=raw_url, domain=domain))
439     return tuple(urls)
440
441
442     def _valid_domain_name(domain: str) -> bool:
443         if len(domain) > 253 or "." not in domain:
444             return False
445         labels = domain.split(".")
446         return all(
447             re.fullmatch(r"[a-z0-9](?:[a-z0-9-]{0,61}[a-z0-9])?", label) is not None for
label in labels
448         )
449
450
451     def _domain_matches_blocked(domain: str, blocked_domain: str) -> bool:
452         return domain == blocked_domain or domain.endswith(f".{blocked_domain}")
453
454
455     class UrlReputationStage:
456         """ADR 0012 Tier 3 captured-URL domain blocklist checks."""
457
458         stage_name = "url-reputation"
459
460         async def evaluate(self, ctx: PolicyContext) -> StageResult:
461             try:
462                 config = load_domain_reputation_config(ctx.policy)
463             except (json.JSONDecodeError, ValueError) as exc:
464                 log.warning(
465                     "domain reputation disabled due to invalid config policy_id=%s",
466                     ctx.policy.id,
467                     exc_info=True,
468                 )
469                 return StageResult.pass_(
470                     self.stage_name,
471                     f"invalid domain reputation config ignored: {exc}",
472                     invalid_domain_reputation_config=True,
473                 )
474
475             if config.invalid_domains:
476                 log.warning(
477                     "domain reputation ignored invalid domains policy_id=%s
invalid_domains=%s",
478                     ctx.policy.id,
479                     list(config.invalid_domains),
480                 )
481             if not config.enabled:
482                 metadata = _url_reputation_metadata((), config)
483                 if config.invalid_domains:
484                     return StageResult.pass_(
485                         self.stage_name,
486                         "domain reputation has no valid blocked domains",
487                         **metadata,
488                     )
489                 return StageResult.pass_(self.stage_name, "domain reputation disabled")
490
491             urls = extract_captions_urls(ctx.content_item.link_url) + extract_captions_urls(
492                 ctx.content_item.text
493             )
494             if not urls:
495                 return StageResult.pass_(

```

```

496         self.stage_name,
497         "no links to scan",
498         **_url_reputation_metadata(urls, config),
499     )
500
501     for url in urls:
502         for blocked_domain in config.blocked_domains:
503             if _domain_matches_blocked(url.domain, blocked_domain):
504                 return StageResult.fail(
505                     self.stage_name,
506                     f"blocked domain matched: {blocked_domain}",
507                     **_url_reputation_metadata(urls, config),
508                     blocked_domain=url.domain,
509                     matched_blocklist_domain=blocked_domain,
510                     matched_url=url.original_url,
511                 )
512
513     return StageResult.pass_(
514         self.stage_name,
515         "links passed domain reputation checks",
516         **_url_reputation_metadata(urls, config),
517     )
518
519

```

## 6. assistant (2026-07-06T23:20:03.719Z)

Now let me look at the caller at telethon\_client.py and run the repro.

## 7. user (2026-07-06T23:20:05.147Z)

```

270     )
271     if channel.id is not None:
272         source = await db.get_source_by_id(conn, channel.id)
273     except Exception:
274         log.warning(
275             "failed to upsert channel %s for user %s", channel_id, user.id,
exc_info=True
276         )
277
278     text = _message_text(message)
279     result = engine_holder[ENGINE_KEY].evaluate(meta)
280     matched = result.matched
281     reasons = list(result.reasons)
282     if matched and source is not None:
283         policy_passed, policy_results = await _evaluate_policy_stages(
284             conn,
285             user.id,
286             source,
287             _content_item_preview(
288                 user.id,
289                 source,
290                 message_id,
291                 document_id,
292                 meta,
293                 text,
294                 matched,
295             ),
296         )
297     if policy_results:
298         reasons.extend(result.reason for result in policy_results)
299         matched = policy_passed
300
301     record = VideoRecord(
302         user_id=user.id,

```

```

303         channel_id=channel_id,
304         message_id=message_id,
305         text=text,
306         document_id=document_id,
307         duration_s=meta.duration_s,
308         width=meta.width,
309         height=meta.height,
310         size_bytes=meta.size_bytes,
311         mime_type=meta.mime_type,
312         matched=matched,
313     )
314     try:
315         inserted = await db.insert_video(conn, record)
316     except sqlite3.IntegrityError:
317         # Lost a dedup race: a concurrent delivery of the same file
318         # (same document_id or same message instance) won the insert.
319         # The three-tier check above already classified this as new, so
320         # the collision means another task is processing (or has
321         # processed) it ? log and skip.
322         log.info(
323             "video_dedup_race user=%s channel=%s msg=%s doc_id=%s",
324             user.id,
325             channel_id,
326             message_id,
327             document_id,
328         )
329     return

```

## 8. assistant (2026-07-06T23:20:08.351Z)

Confirmed: the `\_evaluate\_policy\_stages` call at line 283 is NOT wrapped in try/except, and the record insert at 301-315 follows it. Let me check whether `\_evaluate\_policy\_stages` itself has any exception handling, then run the repro.

## 9. user (2026-07-06T23:20:09.460Z)

```

431:async def _evaluate_policy_stages(
432-    conn: aiosqlite.Connection,
433-    account_id: int,
434-    source: Source,
435-    content_item: ContentItem,
436-) -> tuple[bool, list[StageResult]]:
437-    """Evaluate ADR 0012 policy stages for a candidate item."""
438-    policy = await db.get_default_policy(conn, account_id)
439-    if policy is None:
440-        return True, []
441-    return await evaluate_pipeline(
442-        PolicyContext(content_item=content_item, source=source, policy=policy),
443-        [
444-            TopicPreferenceStage(),
445-            UrlReputationStage(),
446-            CaptionSafetyStage(),
447-            DeepInspectionStage(),
448-        ],
449-    )
450-
451-
452-def _peer_id(peer: object) -> int:
453-    """Best-effort numeric id for the peer of a message.
454-
455-    Telegram represents channel/group/user peers as different TL types; we
456-    collapse them to a single int for dedup/storage. The exact subclass isn't
457-    important ? only that the id is stable per chat.
458-    """
459-    # Each Peer* type has a field like channel_id / chat_id / user_id.

```

```

460-     for field in ("channel_id", "chat_id", "user_id"):
461-         val = getattr(peer, field, None)
462-         if val is not None:
463-             return int(val)
464-     # Fallback: hash the repr so we never collide with real ids (negative).
465-     return -abs(hash(repr(peer)) % (2**31))
466-
467-
468-def _channel_title(event: NewMessage.Event) -> str | None: # pragma: no cover
469-     """Best-effort channel title from the event, if available."""
470-     chat = getattr(event, "chat", None)
471-     return getattr(chat, "title", None) if chat is not None else None
472-
473-
474-class _NewMessageCallback:
475-     """Structural type annotation for the handler returned by make_*_handler."""
476-

```

## 10. assistant (2026-07-06T23:20:12.433Z)

Confirmed: `\_evaluate\_policy\_stages` has no try/except either. Here `PolicyContext` is created without `fetcher`/`budget` (both default None), so the live path fails at "budget missing" ? the finding correctly notes this makes it dormant today. Now let me run the repro to confirm the exception actually propagates.

## 11. assistant (2026-07-06T23:20:25.190Z)

```

import asyncio
from tg_video_filter.policy import (
    DeepInspectionStage, PolicyContext, PolicyBudget, evaluate_pipeline,
)

```

## Inspect the module to build minimal context objects.

```

import inspect
import tg_video_filter.policy as p

```

## Discover ContentItem / Policy shapes

```

print("=== module members ===")
for name in ("ContentItem", "PolicyContext", "PolicyBudget"):
    obj = getattr(p, name, None)
    print(name, obj)

```

```

class BoomFetcher:
    async def inspect(self, content_item, budget):
        raise RuntimeError("ffprobe exploded")

```

```

async def main():
    # Build a ContentItem with size_bytes set small so budget size check passes.
    CI = p.ContentItem
    sig = inspect.signature(CI)
    print("ContentItem params:", list(sig.parameters))

    # Try to build a minimal content item
    try:
        ci = CI(size_bytes=10)
    except TypeError as e:
        print("simple ctor failed:", e)
        # fill required
        kwargs = {}
        for pname, param in sig.parameters.items():
            if param.default is inspect._empty:

```



```

        if pname == "size_bytes":
            kwargs[pname] = 10
        else:
            kwargs[pname] = None
    ci = CI(**kwargs)
    # ensure size_bytes
    try:
        object.__setattr__(ci, "size_bytes", 10)
    except Exception:
        pass
print("content_item:", ci, "size_bytes=", getattr(ci, "size_bytes", "?"))

# Build a policy that enables deep inspection.
Policy = p.Policy if hasattr(p, "Policy") else None
print("Policy:", Policy)
psig = inspect.signature(Policy)
print("Policy params:", list(psig.parameters))

# deep_inspect enabled policy
import json
pkwargs = {}
for pname, param in psig.parameters.items():
    if param.default is inspect._empty:
        pkwargs[pname] = None
# Guess settings field name
print("guessed policy kwargs:", pkwargs)
policy = Policy(**pkwargs)
print("policy:", policy)

# We need load_deep_inspection_enabled(policy) to return True.
# Inspect that function.
print(inspect.getsource(p.load_deep_inspection_enabled))

```

```
asyncio.run(main())
```

## 12. user (2026-07-06T23:20:25.222Z)

File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/repro\_f5.py (file state is current in your context ? no need to Read it back)

## 13. assistant (2026-07-06T23:20:27.401Z)

Rather than guess the shapes, let me look at the existing tests which already construct these objects.

## 14. user (2026-07-06T23:20:29.079Z)

```

14:     DeepInspectionStage,
17:     PolicyBudget,
40:     budget: PolicyBudget | None = None,
43:     content_item=ContentItem(
509:# --- DeepInspectionStage -----
514:         self.calls: list[tuple[ContentItem, PolicyBudget]] = []
516:     async def inspect(self, content_item: ContentItem, budget: PolicyBudget) -> StageResult:
522:     async def inspect(self, content_item: ContentItem, budget: PolicyBudget) -> StageResult:
532:     async def inspect(self, content_item: ContentItem, budget: PolicyBudget) -> StageResult:
542:async def test_deep_inspection_opt_out_passes_without_fetcher_or_budget() -> None:
545:     result = await DeepInspectionStage().evaluate(_ctx(fetcher=fetcher, size_bytes=1_000))
552:async def test_deep_inspection_missing_budget_fails_before_fetcher() -> None:
555:     result = await DeepInspectionStage().evaluate(
556:         _ctx(policy_json='{"deep_inspect":true}', fetcher=fetcher, size_bytes=1_000)
565:async def test_deep_inspection_invalid_budget_fails_before_fetcher() -> None:

```

```

567:     budget = PolicyBudget(max_bytes_per_item=0, timeout_s=1.0, max_concurrent=1)
569:     result = await DeepInspectionStage().evaluate(
571:         policy_json='{"deep_inspect":true}',
589:async def test_deep_inspection_missing_size_fails_before_fetcher() -> None:
591:     budget = PolicyBudget(max_bytes_per_item=2_000, timeout_s=1.0, max_concurrent=1)
593:     result = await DeepInspectionStage().evaluate(
594:         _ctx(policy_json='{"deep_inspect":true}', fetcher=fetcher, budget=budget)
606:async def test_deep_inspection_budget_exceeded_fails_before_fetcher() -> None:
608:     budget = PolicyBudget(max_bytes_per_item=999, timeout_s=1.0, max_concurrent=1)
610:     result = await DeepInspectionStage().evaluate(
612:         policy_json='{"deep_inspect":true}',
630:async def test_deep_inspection_timeout_budget_fails_as_stage_result() -> None:
631:     budget = PolicyBudget(max_bytes_per_item=2_000, timeout_s=0.001, max_concurrent=1)
633:     result = await DeepInspectionStage().evaluate(
635:         policy_json='{"deep_inspect":true}',
651:async def test_deep_inspection_budget_limits_concurrency() -> None:
652:     budget = PolicyBudget(max_bytes_per_item=2_000, timeout_s=1.0, max_concurrent=1)
654:     stage = DeepInspectionStage()
659:         policy_json='{"deep_inspect":true}',
667:         policy_json='{"deep_inspect":true}',
679:async def test_deep_inspection_missing_fetcher_fails_after_budget_passes() -> None:
680:     budget = PolicyBudget(max_bytes_per_item=2_000, timeout_s=1.0, max_concurrent=1)
682:     result = await DeepInspectionStage().evaluate(
683:         _ctx(policy_json='{"deep_inspect":true}', budget=budget, size_bytes=1_000)
691:async def test_deep_inspection_valid_budget_delegates_to_fetcher() -> None:

```

[Showing results with pagination = limit: 40]

## 15. assistant (2026-07-06T23:20:31.516Z)

The test helper `\_ctx` and the fake fetchers give me exactly what I need. Let me read the relevant test scaffolding.

## 16. user (2026-07-06T23:20:33.084Z)

```

30     from tg_video_filter.telethon_client import _evaluate_policy_stages
31
32
33     def _ctx(
34         *,
35         policy_json: str = "{}",
36         source_title: str | None = None,
37         text: str | None = None,
38         size_bytes: int | None = None,
39         fetcher: object | None = None,
40         budget: PolicyBudget | None = None,
41     ) -> PolicyContext:
42         return PolicyContext(
43             content_item=ContentItem(
44                 account_id=111,
45                 source_id=1,
46                 provider_message_id="42",
47                 text=text,
48                 size_bytes=size_bytes,
49                 dedup_key="tg:msg:1:42",
50             ),
51             source=Source(
52                 id=1,
53                 account_id=111,
54                 provider="telegram",
55                 provider_source_id="-1001",
56                 title=source_title,
57             ),
58             policy=Policy(
59                 id=1,

```

### 17. user (2026-07-06T23:20:35.419Z)

```
58         policy=Policy(
59             id=1,
60             account_id=111,
61             name="default",
62             config_json=policy_json,
63             is_default=True,
64         ),
65         fetcher=fetcher, # type: ignore[arg-type]
66         budget=budget,
67     )
68
69
70     class RecordingStage:
71         def __init__(
72             self,
73             name: str,
74             calls: list[str],
75             *,
76             passed: bool = True,
77             reason: str | None = None,
```

### 18. assistant (2026-07-06T23:20:37.206Z)

Now I have everything. Let me write a clean repro using the real test helpers.

### 19. assistant (2026-07-06T23:20:47.323Z)

```
import asyncio
from tg_video_filter.policy import (
    DeepInspectionStage, PolicyContext, PolicyBudget, ContentItem, evaluate_pipeline,
    StageResult,
)
from tg_video_filter.db import Policy, Source

def make_ctx(fetcher, budget):
    return PolicyContext(
        content_item=ContentItem(
            account_id=111, source_id=1, provider_message_id="42",
            text=None, size_bytes=1000, dedup_key="tg:msg:1:42",
        ),
        source=Source(id=1, account_id=111, provider="telegram",
            provider_source_id="-1001", title=None),
        policy=Policy(id=1, account_id=111, name="default",
            config_json='{"deep_inspect":true}', is_default=True),
        fetcher=fetcher, budget=budget,
    )

class BoomFetcher:
    async def inspect(self, content_item, budget):
        raise RuntimeError("ffprobe exploded")

async def main():
    budget = PolicyBudget(max_bytes_per_item=2000, timeout_s=1.0, max_concurrent=1)
    ctx = make_ctx(BoomFetcher(), budget)

    # Direct stage
    try:
        r = await DeepInspectionStage().evaluate(ctx)
        print("stage returned:", r)
    except Exception as e:
```

```

        print(f"RAISED UNCAUGHT (stage): {type(e).__name__} {e}")

# Via pipeline
try:
    passed, results = await evaluate_pipeline(ctx, [DeepInspectionStage()])
    print("pipeline returned:", passed, results)
except Exception as e:
    print(f"RAISED UNCAUGHT (pipeline): {type(e).__name__} {e}")

# Control: TimeoutError IS caught
class SlowFetcher:
    async def inspect(self, content_item, budget):
        await asyncio.sleep(10)
slow_budget = PolicyBudget(max_bytes_per_item=2000, timeout_s=0.001, max_concurrent=1)
ctx2 = make_ctx(SlowFetcher(), slow_budget)
r = await DeepInspectionStage().evaluate(ctx2)
print("timeout control -> stage result:", r.passed, r.reason)

asyncio.run(main())

```

## 20. user (2026-07-06T23:20:47.324Z)

<tool\_use\_error>File has not been read yet. Read it first before writing to it.</tool\_use\_error>

## 21. assistant (2026-07-06T23:20:50.302Z)

The scratchpad file already exists from the earlier write. Let me read then overwrite.

## 22. user (2026-07-06T23:20:50.653Z)

```

1     import asyncio
2     from tg_video_filter.policy import PolicyBudget, DeepInspectionStage, PolicyContext,
StageResult
3     from tg_video_filter.models import ContentItem, Policy, Source
4
5     # Can budget.inspect raise ValueError to the stage? max_concurrent<=0 caught earlier in
_deep_inspection_budget_result.
6     # But what if budget passes the (>0) checks yet _semaphore is None? Only None when
max_concurrent<=0, which is caught. So unreachable. Confirm:
7     b = PolicyBudget(max_bytes_per_item=100, timeout_s=1.0, max_concurrent=0)
8     print("max_concurrent=0 -> _semaphore is None:", b._semaphore is None)
9
10    # size_bytes boundary: exactly max passes to fetcher, max+1 fails
11    async def main():
12        class OKFetcher:
13            async def inspect(self, content_item, budget):
14                return StageResult.pass_("deep-inspection", "delegated")
15        def ctx(size, budget):
16            item = ContentItem.__new__(ContentItem)
17            # build a minimal ContentItem via construction path instead:
18            return size
19        # inspect boundary directly
20        stage = DeepInspectionStage()
21        # Build real objects
22        import json
23        pol = Policy(id=1, account_id=1, name="p", config_json=json.dumps({"deep_inspect":
True}))
24        src = Source.__new__(Source)
25        print("built policy ok")
26
27    asyncio.run(main())
28

```

### 23. assistant (2026-07-06T23:20:53.121Z)

That was from a prior session. Note it imports `ContentItem, Policy, Source` from `tg\_video\_filter.models`. Let me overwrite with my repro.

### 24. assistant (2026-07-06T23:21:01.422Z)

```
import asyncio
from tg_video_filter.policy import (
    DeepInspectionStage, PolicyContext, PolicyBudget, evaluate_pipeline, StageResult,
)
from tg_video_filter.models import ContentItem, Policy, Source

def make_ctx(fetcher, budget):
    return PolicyContext(
        content_item=ContentItem(
            account_id=111, source_id=1, provider_message_id="42",
            text=None, size_bytes=1000, dedup_key="tg:msg:1:42",
        ),
        source=Source(id=1, account_id=111, provider="telegram",
            provider_source_id="-1001", title=None),
        policy=Policy(id=1, account_id=111, name="default",
            config_json='{"deep_inspect":true}', is_default=True),
        fetcher=fetcher, budget=budget,
    )

class BoomFetcher:
    async def inspect(self, content_item, budget):
        raise RuntimeError("ffprobe exploded")

async def main():
    budget = PolicyBudget(max_bytes_per_item=2000, timeout_s=1.0, max_concurrent=1)
    ctx = make_ctx(BoomFetcher(), budget)

    try:
        r = await DeepInspectionStage().evaluate(ctx)
        print("stage returned:", r)
    except Exception as e:
        print(f"RAISED UNCAUGHT (stage): {type(e).__name__} {e}")

    try:
        passed, results = await evaluate_pipeline(ctx, [DeepInspectionStage()])
        print("pipeline returned:", passed, results)
    except Exception as e:
        print(f"RAISED UNCAUGHT (pipeline): {type(e).__name__} {e}")

    # Control: TimeoutError IS caught -> clean fail StageResult
    class SlowFetcher:
        async def inspect(self, content_item, budget):
            await asyncio.sleep(10)
    slow_budget = PolicyBudget(max_bytes_per_item=2000, timeout_s=0.001, max_concurrent=1)
    ctx2 = make_ctx(SlowFetcher(), slow_budget)
    r = await DeepInspectionStage().evaluate(ctx2)
    print("timeout control -> passed=%s reason=%s" % (r.passed, r.reason))

asyncio.run(main())
```

### 25. user (2026-07-06T23:21:01.449Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/repro2.py has been updated successfully.

```

225             channel_id,
226             message_id,
227             existing.channel_id,
228             existing.message_id,
229         )

```

```

230         return
231
232     # Tier 2: metadata fingerprint ? same file re-uploaded (rare but possible).
233     if (
234         meta.size_bytes is not None
235         and meta.duration_s is not None
236         and meta.height is not None
237         and meta.width is not None
238     ):
239         existing = await db.get_video_by_content_fingerprint(
240             conn,
241             user.id,
242             meta.size_bytes,
243             meta.duration_s,
244             meta.height,
245             meta.width,
246         )
247         if existing is not None:
248             log.info(
249                 "video_dedup_fingerprint user=%s size=%s dur=%s h=%s w=%s "
250                 "channel=%s msg=%s original_channel=%s original_msg=%s",
251                 user.id,
252                 meta.size_bytes,
253                 meta.duration_s,
254                 meta.height,
255                 meta.width,
256                 channel_id,
257                 message_id,
258                 existing.channel_id,
259                 existing.message_id,
260             )
261             return
262
263     source = await db.get_source_by_provider_id(conn, user.id, "telegram",
264 str(channel_id))
265
266     # Lazily record the channel (best-effort; failures are non-fatal).
267     try:
268         channel = await db.upsert_channel(
269             conn,
270             Channel(user_id=user.id, channel_id=channel_id,
271 title=_channel_title(event)),

```

### 32. assistant (2026-07-06T23:21:25.412Z)

Let me find the enclosing function definition and any outer handler wrapper.

### 33. user (2026-07-06T23:21:27.598Z)

No matches found

[Showing results with pagination = offset: 50]

### 34. user (2026-07-06T23:21:30.840Z)

```

163: def make_new_message_handler(
184:     async def on_new_message(event: NewMessage.Event) -> None: # pragma: no cover
273:         except Exception:
316:         except sqlite3.IntegrityError:
431: async def _evaluate_policy_stages(
477:     async def __call__(self, event: NewMessage.Event) -> None: ...

```

### 35. user (2026-07-06T23:21:33.652Z)

```

184     async def on_new_message(event: NewMessage.Event) -> None: # pragma: no cover
185         message: Message = event.message # type: ignore[assignment]
186         if message is None:
187             return # Telethon edge case: event has no message (shouldn't happen)
188         # Only process genuine channel/group posts and private messages we
189         # receive ? outgoing messages (sent by our own account) are ignored.
190         if getattr(message, "out", False):
191             return
192
193         meta = extract_video_meta(message)
194         if meta is None:
195             return # not a video
196
197         channel_id = _peer_id(message.peer_id)
198         message_id = message.id
199         document_id = _extract_document_id(message)
200
201         # --- watched gate (ADR 0009, Track E) -----
202         # Opt-in: a source is ingested only when watched=True. An absent
203         # sources row means the channel has never been seen ? treat it as

```

### 36. assistant (2026-07-06T23:21:46.775Z)

The handler `on_new_message` has no top-level try/except wrapping the whole body ? the exception would propagate to Telethon's event dispatcher (which typically logs and continues to next event, not a process crash). The finding's phrasing is "message handler aborts mid-processing" ? accurate: the video is never recorded (insert at 301-315 skipped). It does not claim a process crash. Good ? the scenario is accurately characterized.

Now severity assessment. This is the crux of the adversarial review:

- The finding rates it **medium**. Is it a budget/timeout/concurrency BYPASS (=high)? No. When the fetcher raises, the heavy work has already **failed** ? nothing runs "unbounded". The semaphore and `wait_for` are still in force. So it is not a bypass. High is not warranted.
- Is it a real latent design defect? Yes: the failure direction for non-timeout fetcher errors is unspecified/uncaught, inconsistent with the stage's own timeout handling and the pipeline's fail-open-on-config-error discipline. When a fetcher lands, this determines whether a video is silently dropped-from-processing vs cleanly classified. That is a genuine design gap that should be pinned before a fetcher ships.
- It is **dormant** today (fetcher=None ? fails at "budget missing" first). This pulls toward low. But the finding explicitly acknowledges the dormancy and correctly notes it activates the moment a real fetcher is added ? which is the stated purpose of this boundary PR. A latent-but-inevitable design gap in the exact code this PR introduces is a legitimate medium, not merely a hygiene note.

Let me double-check one refutation angle: does `PolicyBudget.inspect`'s `wait_for` convert a fetcher's arbitrary exception into something else? `asyncio.wait_for` re-raises the inner exception unchanged (only wraps timeout). My repro already confirmed `RuntimeError` passes through verbatim. Refutation fails.

The finding is accurate in code facts, mechanism, dormancy caveat, and remediation. Severity medium is correct (it is a real latent design defect, not a bypass and not merely hygiene).

### 37. user (2026-07-06T23:21:58.223Z)

Structured output provided successfully